CS462
Machine Learning
Nov 2024

Project: Date Fruit Classification

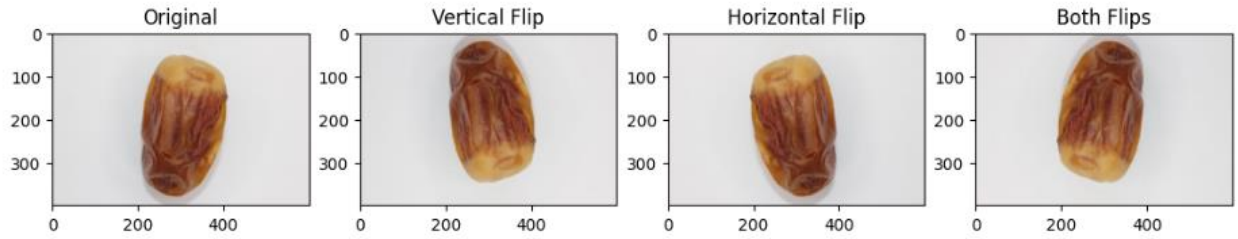
Talal Aloushan - 441102306

Problem description and its importance:
In today's fast-paced agricultural industry, telling different types of dates apart quickly and accurately is a real challenge. Imagine having to sort through thousands of dates by hand - it's time-consuming and prone to mistakes, even for experienced workers. That's where modern technology comes in handy, specifically computer vision. By teaching computers to "see" and categorize dates automatically, we can transform this tedious task into a streamlined process. Think about it - each variety of date has its own special characteristics, commanding different prices in the market and being preferred for different uses. Some dates are perfect for luxury gift boxes, while others are ideal for processing into date syrup or paste. By using artificial intelligence to classify these dates automatically, we're not just saving time - we're ensuring consistent quality control, accurate pricing, and proper sorting. This means farmers get fair prices for their premium varieties, processors get the right types for their products, and consumers can trust what they're buying.

Platform:
Jupyter Notebook

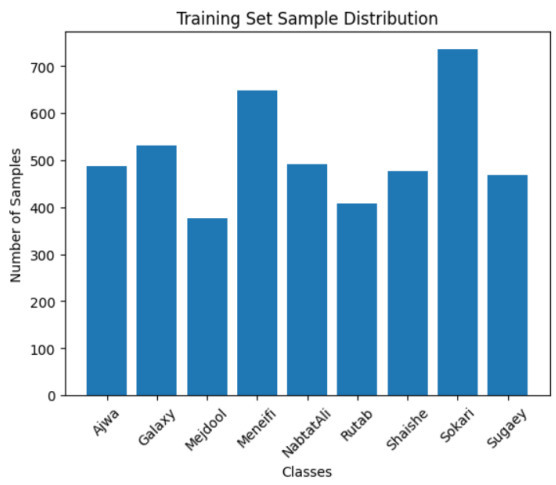
Dataset:
Training set: 1156 samples
Validation set: 502 samples
Size: 600x400

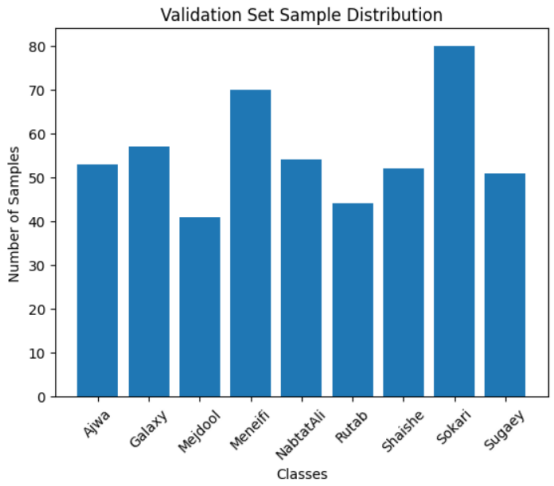
Augmentation for training set:



New training set samples: 4624

Count per label charts:





CNN Approaches:

I took 3 approaches to train a CNN model:

1: built a custom CNN using tensorflow framework and trained it. (94% acc)

2: applied the transfer learning concept on tf effecientnetb0 in 2 ways.

- 2.1:froze all the layers and weights, added an output layer and trained it. (96%)
- 2.2:unfroze the layers and fine-tuned the model. (97%)

3: fine-tuned yolov11n-cls pytorch model. (99%)

Approach #1 (Custom CNN model)

Data preprocessing:

Batched data into 32 batches.

Scaled pixel values to [0,1].

Resized images into 200x200.

CNN Architecture:

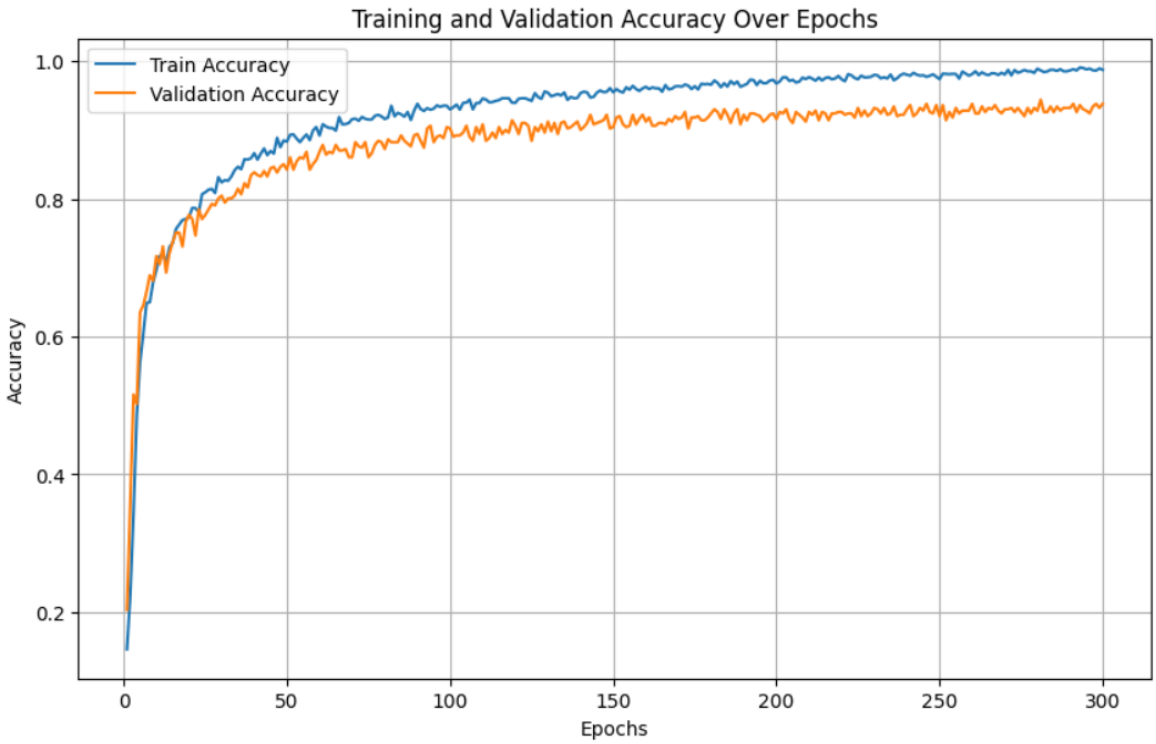
```
┌──────────────────────────────────────────────────────────┬──────────────────────────┬──────────────────────
──────┬──────────────────────────┐
| Layer (type)                                             | Output Shape             |              Param #
|
```

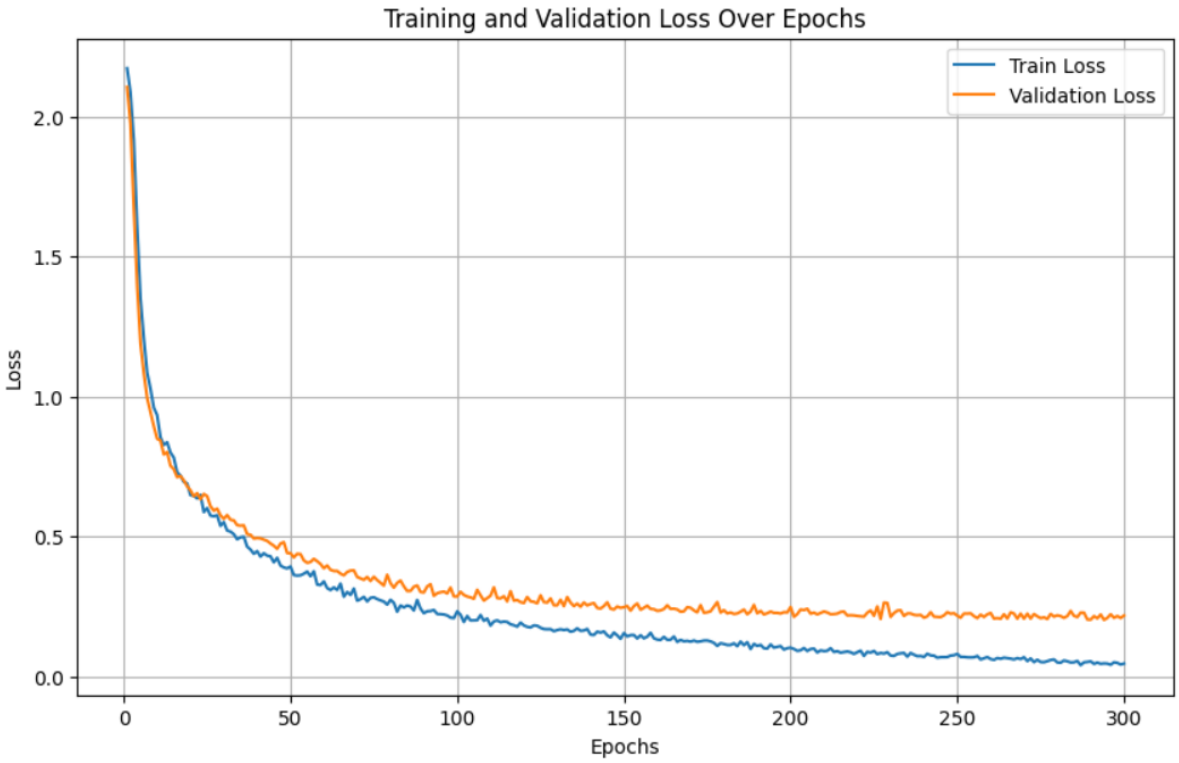
conv2d_9 (Conv2D)	(None, 196, 196, 32)	2,432
max_pooling2d_9 (MaxPooling2D)	(None, 98, 98, 32)	0
conv2d_10 (Conv2D)	(None, 96, 96, 32)	9,248
max_pooling2d_10 (MaxPooling2D)	(None, 48, 48, 32)	0
conv2d_11 (Conv2D)	(None, 46, 46, 32)	9,248
max_pooling2d_11 (MaxPooling2D)	(None, 23, 23, 32)	0
flatten_3 (Flatten)	(None, 16928)	0
dense_6 (Dense)	(None, 256)	4,333,824
dropout_3 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 9)	2,313

Total params: 4,357,065 (16.62 MB)

Optimizer and Epochs:
Adam, learning rate: 0.00001.
Loss Function: Categorical Crossentropy.
Metrics: Accuracy.
Epochs: 300.

Loss and accuracy plots:



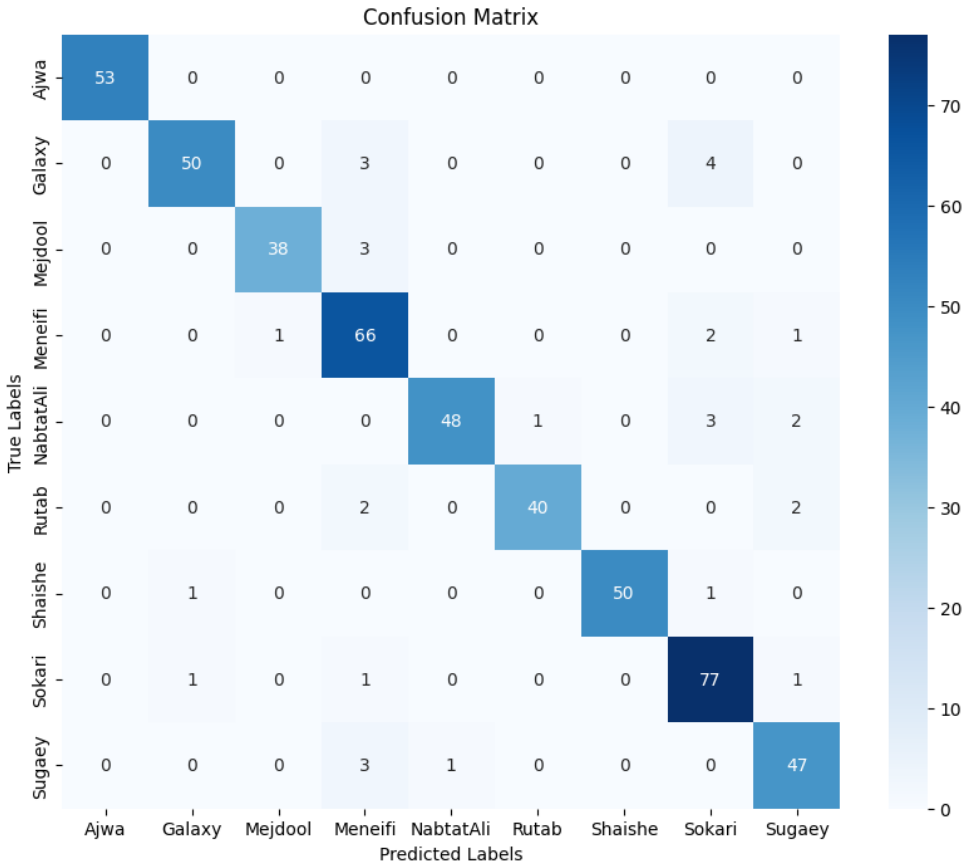


Evaluation results:

classification report:

	precision	recall	f1-score	support
Ajwa	1.00	1.00	1.00	53
Galaxy	0.96	0.88	0.92	57
Mejdool	0.97	0.93	0.95	41
Meneifi	0.85	0.94	0.89	70
NabtatAli	0.98	0.89	0.93	54
Rutab	0.98	0.91	0.94	44
Shaishe	1.00	0.96	0.98	52
Sokari	0.89	0.96	0.92	80
Sugaey	0.89	0.92	0.90	51
accuracy			0.93	502
macro avg	0.95	0.93	0.94	502
weighted avg	0.94	0.93	0.93	502

confusion matrix:



Screenshots:

Source code:

```python
# Imports
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
import os
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
from tensorflow.keras.optimizers import Adam
```

```python
# paths
train_dir = 'Date_Fruit/train'
val_dir = 'Date_Fruit/test'

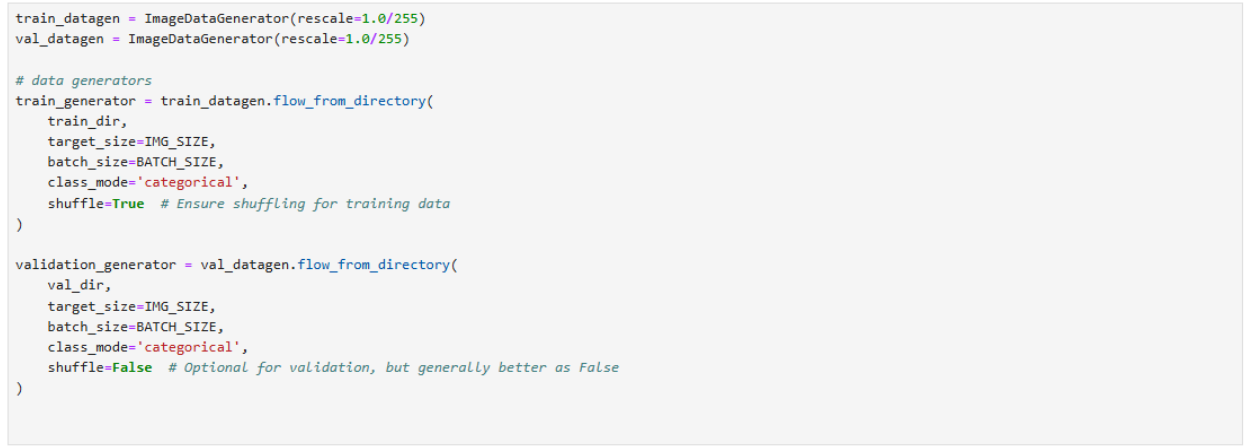
# parameters
IMG_SIZE = (200, 200)
BATCH_SIZE = 32
```

```python
# Data visualization: Count samples per label
def plot_label_distribution(directory, title):
    labels = []
    for subdir in os.listdir(directory):
        path = os.path.join(directory, subdir)
        if os.path.isdir(path):
            labels.append(len(os.listdir(path)))

    plt.bar(os.listdir(directory), labels)
    plt.title(title)
    plt.xlabel('Classes')
    plt.ylabel('Number of Samples')
    plt.xticks(rotation=45)
    plt.show()

# Plot for training set
plot_label_distribution(train_dir, "Training Set Sample Distribution")

# Plot for validation set
plot_label_distribution(val_dir, "Validation Set Sample Distribution")
```



```python
train_datagen = ImageDataGenerator(rescale=1.0/255)
val_datagen = ImageDataGenerator(rescale=1.0/255)

# data generators
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True  # Ensure shuffling for training data
)

validation_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False  # Optional for validation, but generally better as False
)
```

```
Found 4624 images belonging to 9 classes.
Found 502 images belonging to 9 classes.
```

```python
#best for now, incremental learning rate, can achieve high val accuarcy with higer eopchs, no overfitting
model = models.Sequential([
    layers.Conv2D(32, (5, 5), activation='relu', input_shape=(200, 200, 3)), # was 400x400
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(9, activation='softmax')
])

model.summary()
```

```
optimizer = Adam(learning_rate=0.00001) # was 0.00001

model.compile(
    optimizer=optimizer,
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    train_generator,
    epochs=500,
    validation_data=validation_generator
)
```

```
# Evaluate the model
loss, accuracy = model.evaluate(validation_generator)
print(f"Validation Loss: {loss}")
print(f"Validation Accuracy: {accuracy}")

# Plot training history
# Accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Model Accuracy')
plt.show()

# Loss
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Model Loss')
plt.show()
```

```
# Confusion Matrix and Classification Report
# Get true labels and predictions
true_labels = validation_generator.classes
class_labels = list(validation_generator.class_indices.keys())

# Predict
predictions = model.predict(validation_generator)
predicted_classes = np.argmax(predictions, axis=1)

# Classification Report
print("Classification Report:")
print(classification_report(true_labels, predicted_classes, target_names=class_labels))

# Confusion Matrix
conf_matrix = confusion_matrix(true_labels, predicted_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=class_labels, yticklabels=class_labels)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.show()
```

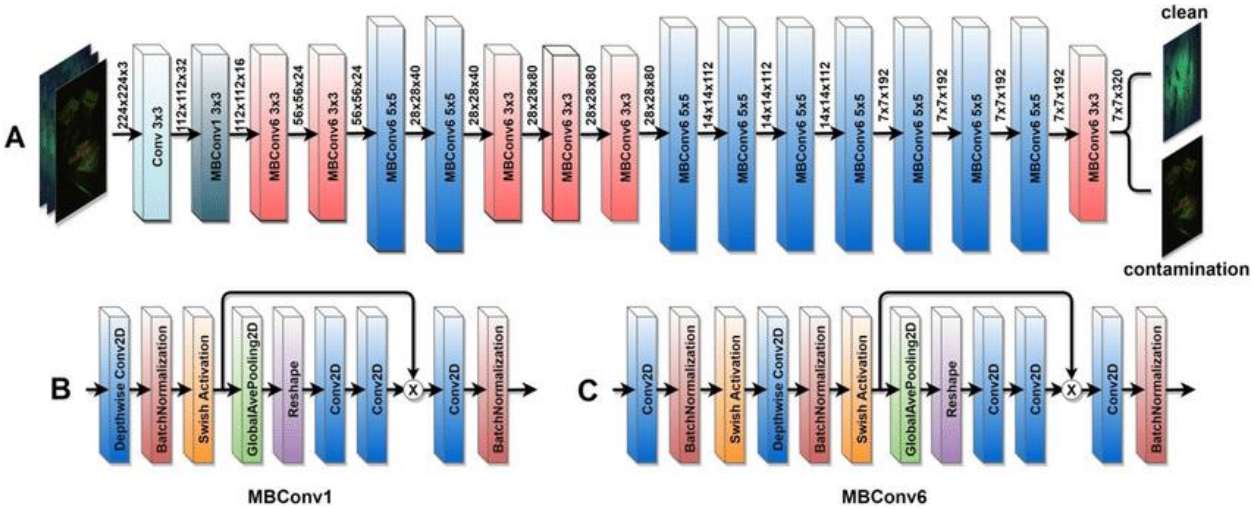
Evaluation results:

```
145/145 ━━━━━━━━━━━━━━━━━━━━ 49s 338ms/step - accuracy: 0.9873 - loss: 0.0519 - val_accuracy: 0.9263 - val_loss: 0.2236
Epoch 274/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 342ms/step - accuracy: 0.9833 - loss: 0.0603 - val_accuracy: 0.9323 - val_loss: 0.2155
Epoch 275/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 341ms/step - accuracy: 0.9850 - loss: 0.0522 - val_accuracy: 0.9263 - val_loss: 0.2089
Epoch 276/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 339ms/step - accuracy: 0.9874 - loss: 0.0517 - val_accuracy: 0.9203 - val_loss: 0.2118
Epoch 277/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9854 - loss: 0.0552 - val_accuracy: 0.9323 - val_loss: 0.2060
Epoch 278/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9855 - loss: 0.0594 - val_accuracy: 0.9243 - val_loss: 0.2240
Epoch 279/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 339ms/step - accuracy: 0.9831 - loss: 0.0615 - val_accuracy: 0.9323 - val_loss: 0.2144
Epoch 280/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9893 - loss: 0.0507 - val_accuracy: 0.9283 - val_loss: 0.2190
Epoch 281/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9866 - loss: 0.0498 - val_accuracy: 0.9442 - val_loss: 0.2162
```

Approach #2 (EffecientnetB0)

Data preprocessing:
Batched data into 32 batches.
Scaled pixel values to [0,1].
Resized images into 224x224.

EffecientnetB0 Architecture:



Frozen layers + a pooling, a drop out and my output dense layer :

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 9)	11,529

Total params: 4,061,100 (15.49 MB)

Trainable params: 11,529 (45.04 KB)

Optimizer and Epochs(frozen model):
Adam, learning rate: 0.01.
Loss Function: Categorical Crossentropy.
Metrics: Accuracy.
Epochs: 20.

Optimizer and Epochs(unfrozen model):
Adam, learning rate: 0.0001.
Loss Function: Categorical Crossentropy.
Metrics: Accuracy.
Epochs: 20.

Screenshots:

Source code:

```python
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input

# Constants
batch_size = 32
IMG_WIDTH = 224
IMG_HEIGHT = 224
CHANNELS = 3
CLASSES = 9
train_dir = 'Date_Fruit/train'
validation_dir = 'Date_Fruit/test'



print(tf.keras.__version__)


train_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)
validation_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

# Load Training Data
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_WIDTH, IMG_HEIGHT),
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=True
)

# Load Validation Data
validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=(IMG_WIDTH, IMG_HEIGHT),
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)

# Create Model
model_ENB0 = EfficientNetB0(include_top=False, weights='imagenet', input_shape=(IMG_WIDTH, IMG_HEIGHT, CHANNELS))
model_ENB0.trainable = False  # Freeze the pre-trained base model
```

```
model = Sequential([
    model_ENB0,
    GlobalAveragePooling2D(),
    Dropout(0.2),
    Dense(CLASSES, activation="softmax")
])

model.summary()

# Compile Model
model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=1e-2),
    metrics=['accuracy']
)

# Training Parameters
STEP_SIZE_TRAIN = train_generator.n // train_generator.batch_size
STEP_SIZE_VALID = validation_generator.n // validation_generator.batch_size
epochs = 20

# Train the Model
history = model.fit(
    train_generator,
   # steps_per_epoch=STEP_SIZE_TRAIN,
    epochs=epochs,
    validation_data=validation_generator,
   # validation_steps=STEP_SIZE_VALID
)

# Plot Training History
def plot_training_history(history):
    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    loss = history.history['loss']
    val_loss = history.history['val_loss']

    plt.figure(figsize=(12, 4))

    # Accuracy
    plt.subplot(1, 2, 1)
    plt.plot(acc, label='Training Accuracy')
    plt.plot(val_acc, label='Validation Accuracy')
    plt.legend()
    plt.title('Accuracy')

    # Loss
    plt.subplot(1, 2, 2)
    plt.plot(loss, label='Training Loss')
    plt.plot(val_loss, label='Validation Loss')
    plt.legend()
    plt.title('Loss')
```

```
# Unfreezing
model_ENB0.trainable = True
set_trainable = False

for layer in model_ENB0.layers:
    if layer.name == 'block6d_se_excite':
        set_trainable = True
    if set_trainable:
        if not isinstance(layer, tf.keras.layers.BatchNormalization):  # Correct reference
            layer.trainable = True
        else:
            layer.trainable = False
    else:
        layer.trainable = False

# Updated Model Summary
model.summary()

# Fine-Tuning
model.compile(
    loss='categorical_crossentropy',
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),  # Correct reference
    metrics=['accuracy']
)

# Training Parameters
STEP_SIZE_TRAIN = train_generator.n // train_generator.batch_size
STEP_SIZE_VALID = validation_generator.n // validation_generator.batch_size
epochs = 50

# Train the Model
history = model.fit(
    train_generator,
   # steps_per_epoch=STEP_SIZE_TRAIN,
    epochs=epochs,
    validation_data=validation_generator,
  #  validation_steps=STEP_SIZE_VALID
)

# Plot Accuracy
plt.style.use('fivethirtyeight')
plt.title('Model Accuracy')
plt.plot(range(1, epochs + 1), history.history['accuracy'], label="Training Accuracy")
plt.plot(range(1, epochs + 1), history.history['val_accuracy'], label="Validation Accuracy")
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# Plot Loss
plt.style.use('fivethirtyeight')
plt.title('Model Loss')
```

Evaluation results for frozen model:

```
145/145 ━━━━━━━━━━━━━━━━━━━━ 128s 879ms/step - accuracy: 0.9645 - loss: 0.1095 - val_accuracy: 0.9522 - val_loss: 0.1374
Epoch 12/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 151s 1s/step - accuracy: 0.9651 - loss: 0.1004 - val_accuracy: 0.9482 - val_loss: 0.2386
Epoch 13/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 152s 1s/step - accuracy: 0.9653 - loss: 0.1077 - val_accuracy: 0.9582 - val_loss: 0.1535
Epoch 14/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 973ms/step - accuracy: 0.9681 - loss: 0.0954 - val_accuracy: 0.9382 - val_loss: 0.2326
Epoch 15/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 141s 970ms/step - accuracy: 0.9636 - loss: 0.0944 - val_accuracy: 0.9402 - val_loss: 0.2057
Epoch 16/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 141s 970ms/step - accuracy: 0.9621 - loss: 0.1251 - val_accuracy: 0.9422 - val_loss: 0.2029
Epoch 17/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 974ms/step - accuracy: 0.9606 - loss: 0.1246 - val_accuracy: 0.9482 - val_loss: 0.2323
Epoch 18/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 976ms/step - accuracy: 0.9703 - loss: 0.0813 - val_accuracy: 0.9622 - val_loss: 0.1516
Epoch 19/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 976ms/step - accuracy: 0.9710 - loss: 0.0989 - val_accuracy: 0.9522 - val_loss: 0.1929
```

Evaluation results for unfrozen model:

```
Epoch 13/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 162s 1s/step - accuracy: 0.9940 - loss: 0.0170 - val_accuracy: 0.9641 - val_loss: 0.1968
Epoch 14/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 164s 1s/step - accuracy: 0.9964 - loss: 0.0104 - val_accuracy: 0.9701 - val_loss: 0.1711
Epoch 15/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 162s 1s/step - accuracy: 0.9951 - loss: 0.0091 - val_accuracy: 0.9681 - val_loss: 0.1719
Epoch 16/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 157s 1s/step - accuracy: 0.9935 - loss: 0.0139 - val_accuracy: 0.9661 - val_loss: 0.1798
Epoch 17/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 159s 1s/step - accuracy: 0.9942 - loss: 0.0178 - val_accuracy: 0.9622 - val_loss: 0.1857
Epoch 18/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 157s 1s/step - accuracy: 0.9929 - loss: 0.0206 - val_accuracy: 0.9741 - val_loss: 0.1592
Epoch 19/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 160s 1s/step - accuracy: 0.9954 - loss: 0.0102 - val_accuracy: 0.9661 - val_loss: 0.1812
Epoch 20/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 162s 1s/step - accuracy: 0.9974 - loss: 0.0077 - val_accuracy: 0.9641 - val_loss: 0.1728
```

Approach #3 (Yolo11n-cls)

Data preprocessing:
Batched data into 16 batches.
Scaled pixel values to [0,1].
Resized images into 224x224.

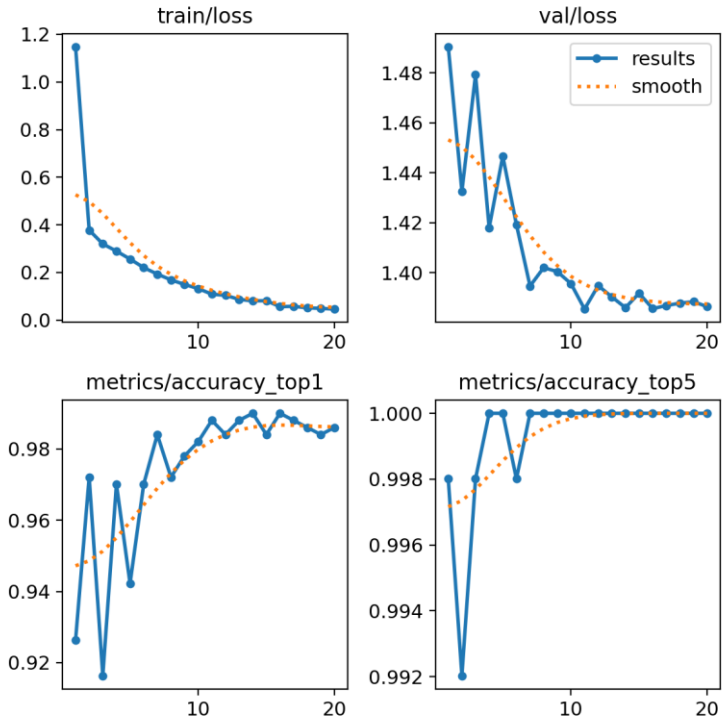
Yolov11n-cls Architecture:

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]

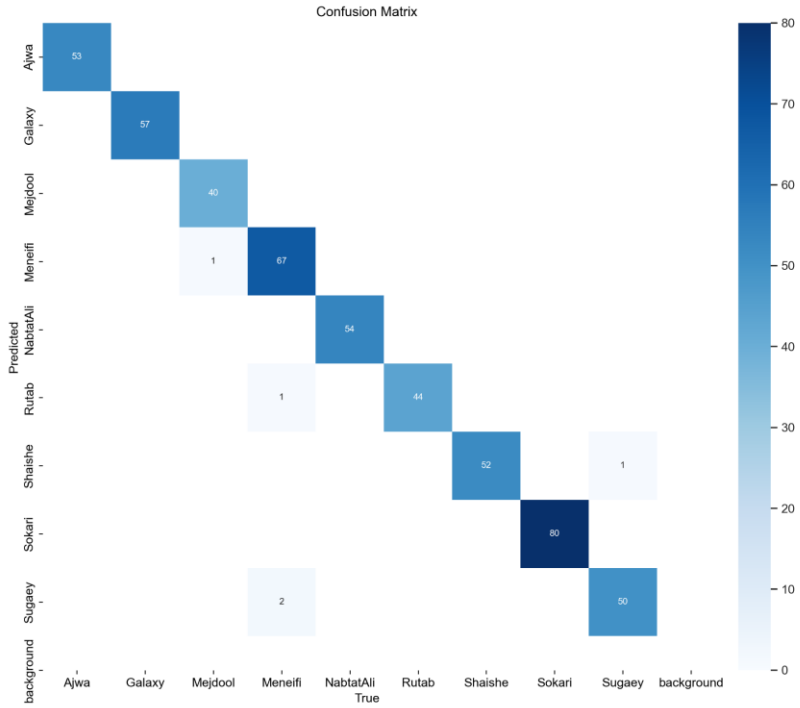
5 -1 1 147712 ultralytics.nn.modules.conv.Conv [128, 128, 3, 2]
6 -1 1 87040 ultralytics.nn.modules.block.C3k2 [128, 128, 1, True]
7 -1 1 295424 ultralytics.nn.modules.conv.Conv [128, 256, 3, 2]
8 -1 1 346112 ultralytics.nn.modules.block.C3k2 [256, 256, 1, True]
9 -1 1 249728 ultralytics.nn.modules.block.C2PSA [256, 256, 1]
10 -1 1 341769 ultralytics.nn.modules.head.Classify [256, 9]
YOLO11n-cls summary: 151 layers, 1,542,633 parameters, 1,542,633 gradients, 3.3 GFLOPs

Optimizer and Epochs:
Adam, learning rate: 0.01.
Loss Function: Categorical Crossentropy.
Metrics: Accuracy.
Epochs: 20.

Loss and accuracy plots:



Evaluation results:
confusion matrix:



Screenshots:

Evaluation results:

```
      Epoch    GPU_mem       loss  Instances       Size
      16/20         0G    0.05629         16        224: 100%|██████████| 289/289 [01:43<00:00,  2.80it/s]
                 classes   top1_acc   top5_acc: 100%|██████████| 16/16 [00:04<00:00,  3.56it/s]
                     all       0.99          1

      Epoch    GPU_mem       loss  Instances       Size
      17/20         0G    0.05785         16        224: 100%|██████████| 289/289 [01:43<00:00,  2.79it/s]
                 classes   top1_acc   top5_acc: 100%|██████████| 16/16 [00:04<00:00,  3.51it/s]
                     all      0.988          1

      Epoch    GPU_mem       loss  Instances       Size
      18/20         0G    0.05155         16        224: 100%|██████████| 289/289 [01:43<00:00,  2.79it/s]
                 classes   top1_acc   top5_acc: 100%|██████████| 16/16 [00:04<00:00,  3.32it/s]
                     all      0.986          1

      Epoch    GPU_mem       loss  Instances       Size
      19/20         0G    0.04915         16        224: 100%|██████████| 289/289 [03:10<00:00,  1.52it/s]
                 classes   top1_acc   top5_acc: 100%|██████████| 16/16 [00:10<00:00,  1.56it/s]
                     all      0.984          1

      Epoch    GPU_mem       loss  Instances       Size
      20/20         0G    0.04634         16        224: 100%|██████████| 289/289 [04:25<00:00,  1.09it/s]
                 classes   top1_acc   top5_acc: 100%|██████████| 16/16 [00:09<00:00,  1.62it/s]
                     all      0.986          1

20 epochs completed in 0.683 hours.
Optimizer stripped from runs\classify\train3\weights\last.pt, 3.2MB
Optimizer stripped from runs\classify\train3\weights\best.pt, 3.2MB

Validating runs\classify\train3\weights\best.pt...
Ultralytics 8.3.31 🚀 Python-3.12.7 torch-2.5.1+cpu CPU (12th Gen Intel(TM) i7-1255U)
YOLO11n-cls summary (fused): 112 layers, 1,537,553 parameters, 0 gradients, 3.2 GFLOPs
WARNING ⚠️ Dataset 'split=val' not found, using 'split=test' instead.
train: C:\Users\HP\yolo11-cls\Date_Fruit\train... found 4624 images in 9 classes ✅
val: None...
test: C:\Users\HP\yolo11-cls\Date_Fruit\test... found 502 images in 9 classes ✅
                 classes   top1_acc   top5_acc: 100%|██████████| 16/16 [00:09<00:00,  1.67it/s]
                     all       0.99          1
```

Feature extraction examples of the model:

